TRAVEL PLANNER

PROGRAMMING WITH PYTHON

(202044504)

PYTHON MINI PROJECT REPORT

A Python application for trip planning, itinerary generation and budget estimation

PROJECT FEATURES	PYTHON CONCEPTS
Destination guide	Dictionaries and lists
Day-wise itinerary	Functions and loops
Trip budget estimate	Input validation
Save plan to text file	File handling and datetime

SUBMITTED BY

KRISHIL PATEL

Enrollment No.: _____

B.Tech. (Information Technology / Computer Science) • Semester: _____

Batch: _____ | Academic Year: 2026–27

A D Patel Institute of Technology, New Vallabh Vidyanagar
Vitthal Udyognagar – 388121, Anand (Gujarat), India

CERTIFICATE

This is to certify that **Krishil Patel**, a student of the Bachelor of Technology programme, has prepared the mini project report entitled **“Travel Planner”** for the course **“Programming with Python” (Course Code: 202044504)**.

The project demonstrates the practical application of core Python programming concepts, including functions, dictionaries, lists, loops, conditional statements, input validation, exception handling, date and time handling, and file operations. The report is submitted in partial fulfilment of the academic requirements of the course.

The Travel Planner is an educational prototype that helps users organize a trip by selecting a destination, entering travel details, generating a simple itinerary and estimating expenses. It does not book travel or retrieve live prices.

Date: _____

Place: _____

Course Faculty

Name: _____

Signature: _____

1. Abstract

Travel Planner is a Python-based mini project designed to help users organize a basic trip in one place. Users can choose a destination, enter the duration and number of travellers, specify travel and accommodation costs, and select interests. The program generates a simple day-wise itinerary and calculates the estimated total budget and cost per person. The plan can be displayed and saved to a text file. The project demonstrates functions, dictionaries, lists, loops, validation, exception handling and file handling.

2. Introduction

Planning a trip involves deciding where to go, choosing attractions, arranging activities and estimating expenses. Doing these tasks separately can be inconvenient, especially when a traveller wants to compare the expected cost with a fixed budget. This project brings the basic steps together in a simple Python application. It is intended for learning and demonstration rather than live booking or professional travel management.

3. Problem Statement

Manual trip planning may require repeated calculations, handwritten schedules and separate notes for expenses. It can be difficult to distribute activities across the available days and estimate the cost per traveller consistently. The Travel Planner addresses these needs by collecting trip details, preparing a day-wise outline and producing a budget summary in a readable format.

4. Objectives

- To develop a practical application using core Python concepts.
- To display sample information about popular Indian destinations and attractions.
- To generate a simple day-wise itinerary based on trip duration and interests.
- To estimate transport, accommodation, food, activities and additional expenses.
- To calculate the overall trip cost and estimated cost per traveller.
- To save the generated travel plan to a text file for later reference.
- To demonstrate functions, collections, validation and file handling.

5. Scope

The project supports six sample destinations: Goa, Jaipur, Manali, Mumbai, Delhi and Udaipur. It uses user-entered transport and accommodation costs with illustrative estimates for food and local activities. It is suitable for a classroom demonstration or basic personal planning. It does not make reservations, process payments, or fetch live fares, hotel prices, weather or availability. Estimates should therefore be treated as examples, not quotations.

6. Hardware and Software Requirements

Component	Requirement
Hardware	Desktop or laptop with keyboard, display and sufficient memory to run Python.
Operating system	Windows, macOS or Linux.
Language	Python 3.8 or later.
Libraries	Python standard library; no external package is required for the basic version.
Editor	Visual Studio Code, IDLE or any Python editor.
File storage	Local text file used to save generated travel plans.

7. System Design

The program can be understood as three logical layers. The input and presentation layer collects the destination, travel duration, number of travellers, transport cost and accommodation details. The application layer validates the values, selects attractions, builds the itinerary and calculates the estimated budget. The file layer writes the completed plan to a local text file when the user chooses to save it. Keeping these responsibilities in separate functions makes the program easier to understand and maintain.

8. Main Modules

- **Destination guide:** stores sample destination names and attractions in dictionaries.
- **Trip input module:** collects destination, days, group size, costs and interests.
- **Itinerary module:** distributes selected attractions across the trip days.
- **Budget module:** calculates estimated expenses and per-person cost.
- **Display module:** formats the itinerary and budget summary for the user.
- **File handling module:** saves the plan to a text file for later reference.
- **Validation module:** checks numeric input and prevents invalid values such as zero travellers or negative costs.

9. Destination Data (Sample)

Destination	Example attractions / experiences
Goa	Beaches, forts, local markets and coastal viewpoints
Jaipur	Amber Fort, City Palace and Hawa Mahal
Manali	Mountain viewpoints, local markets and nature walks
Mumbai	Gateway of India, Marine Drive and museums
Delhi	Red Fort, India Gate and historical sites
Udaipur	City Palace, Lake Pichola and old-city walks

10. Python Concepts Used

- **Variables and data types:** store destination, days, travellers and cost values.
- **Lists:** keep a collection of attractions and itinerary entries.
- **Dictionaries:** map destinations to attractions and sample estimates.
- **Functions:** separate input, validation, itinerary, calculation, display and save operations.
- **Loops:** repeat menu options and create one itinerary entry for each travel day.
- **Conditional statements:** select destination-specific data and validate choices.
- **Exception handling:** handle invalid numeric input and file errors without abruptly terminating the program.
- **File handling:** write the final plan to a text file.
- **Datetime:** record the date and time when a plan is generated.

11. Algorithm

Start the program and display the main menu.

Allow the user to view destination information or create a new plan.

Collect destination, trip duration, number of travellers, interests and expense details.

Validate the inputs; ask again if days, travellers or costs are invalid.

Retrieve sample attractions for the selected destination.

Generate a day-wise itinerary using the available attractions and selected interests.

Calculate estimated food/local activity costs, accommodation, transport and extra budget.

Calculate the total estimated cost and cost per person.

Display the itinerary and budget summary.

Ask whether the user wants to save the plan; write it to a text file if requested.

Return to the menu or exit the program.

12. Budget Calculation

Food and local activities = sample daily estimate × number of days × number of travellers

Accommodation = cost per night × (days – 1) × number of rooms

Total estimated cost = food/local activities + accommodation + transport + extra budget

Cost per person = total estimated cost ÷ number of travellers

All rates are illustrative values for a classroom project. Actual expenses depend on dates, season, transport choice, accommodation, personal preferences and availability. The program does not fetch live prices.

13. Testing

Test case	Input / action	Expected result	Status
Open program	Launch the Python file	Main menu appears	Run locally
View destinations	Choose destination guide	Sample destinations and attractions are displayed	Run locally
Create valid plan	Enter valid trip details	Itinerary and budget summary are generated	Run locally
Invalid duration	Enter text or 0 for days	Validation message requests a valid positive number	Run locally
Invalid group size	Enter 0 or a negative number of travellers	Invalid value is rejected	Run locally
Save a plan	Choose save option	Travel plan is written to a text file	Run locally
File error	Use an unavailable or restricted path	Helpful error message is displayed	Run locally

Testing should be performed on the student's own computer after saving the Python program. The status column is a verification checklist; it does not claim that the tests have already been executed on a particular computer.

14. Exception Handling and Data Safety

The program checks that numeric inputs are valid and that the number of days and travellers is positive. It should also reject negative expense values and handle file-writing errors using try-except blocks. Travel plans are saved locally, so users should choose a suitable folder and avoid storing unnecessary personal information in shared files.

15. Limitations and Future Enhancements

- Add a graphical user interface using Tkinter.
- Add destination search, category filters and user-defined attractions.
- Include editable hotel, transport and activity estimates.
- Add maps, route suggestions and distance calculations.
- Integrate weather forecasts and live price information through suitable APIs.
- Add user accounts and a history of saved trips.
- Export travel plans to PDF or CSV.
- Add calendar integration and optional booking links.

Live information and bookings would require external services and additional validation, security and testing.

16. Conclusion

The Travel Planner mini project provides a practical example of using Python to organize a basic trip. It combines destination data, itinerary generation, budget estimation and text-file saving in one workflow. Through dictionaries, lists, functions, loops, conditional statements, input validation and file handling, the project demonstrates how core programming concepts can solve a familiar real-world problem. The application is an educational prototype and can be extended with a graphical interface, live travel information and more advanced planning features.

17. References

- Python documentation: https://docs.python.org/3/
- Python tutorial: https://docs.python.org/3/tutorial/
- Python file input/output: https://docs.python.org/3/tutorial/inputoutput.html
- Python datetime documentation: https://docs.python.org/3/library/datetime.html

THANK YOU